UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Experiment 8

Student Name: Sumanyu Seth **UID: 25MCI10040**

Branch: MCA (AI & ML) **Section/Group: 25MAM_KAR-1_A**

Semester: II **Date of Performance: 24/02/26**

Subject Name: Technical Training **Subject Code: 25CAP-652**

Experiment Aim: To apply the concept of Stored Procedures in database operations in order to perform tasks like insertion, updating, deletion, and retrieval of data efficiently, securely, and in a reusable manner within the database system.

Objectives: Apply stored procedure concepts for database operations.

Experiment Steps:

1. **Created Employees Table:**

	emp_id [PK] integer	emp_name character varying (50)	manager_id integer	department character varying (50)	salary integer
1	1	Amit	[null]	Management	120000
2	2	Ravi	1	Engineering	80000
3	3	Neha	1	Engineering	82000
4	4	Karan	2	Engineering	60000
5	5	Simran	2	Engineering	62000
6	6	Pooja	3	Engineering	61000
7	7	Rahul	3	Engineering	64000
8	8	Arjun	1	HR	70000

2. **Inserted Stored procedure:**

```
CREATE OR REPLACE PROCEDURE add_employee(

p_id INT,

p_name VARCHAR,

p_manager INT,
```

```
p_dept VARCHAR,

p_salary INT

)

LANGUAGE plpgsql

AS $$

BEGIN

INSERT INTO Employees VALUES (p_id, p_name, p_manager, p_dept, p_salary);

END;

$$;
```

Call:

```
CALL add_employee(9, 'Ankit', 2, 'Engineering', 65000);
```

3. Update stored procedure:

```
create or replace Procedure UPDATE_SALARY_PROCC(IN P_EMP_ID int, INOUT P_SALARY
numeric(20,3) , OUT STATUS

varchar(20))

AS

$$

DECLARE

CURR_SAL NUMERIC(20,3);

BEGIN

SELECT SALARY+P_SALARY INTO CURR_SAL FROM employees WHERE EMP_ID = P_EMP_ID;

IF NOT FOUND THEN

RAISE EXCEPTION 'EMPLOYEE NOT FOUND';

END IF ;

UPDATE employees

SET SALARY = CURR_SAL WHERE EMP_ID = P_EMP_ID;

P_SALARY:= CURR_SAL;
```

```
STATUS :='SUCCESS';

EXCEPTION

WHEN OTHERS THEN

IF SQLERRM LIKE '%EMPLOYEE NOT FOUND%' THEN

STATUS:='EMPLOYEE NOT FOUND';

END IF;

END;

$$ LANGUAGE PLPGSQL ;
```

Call:

```
DO

$$

DECLARE

EMP_ID INT :=3;

STATUS VARCHAR(20);

SALARY NUMERIC(20,3):=525;

BEGIN

CALL UPDATE_SALARY_PROCC(EMP_ID ,SALARY , STATUS);

RAISE NOTICE 'YOUR STATUS IS % AND THE UPDATED SALARY IS %',STATUS , SALARY;

END;

$$
```

```
NOTICE:  YOUR STATUS IS SUCCESS AND THE UPDATED SALARY IS 82525.000
DO
```

4. Delete stored procedure:

```
CREATE OR REPLACE PROCEDURE delete_employee(

p_id INT

)

LANGUAGE plpgsql
```

```
AS $$

BEGIN

DELETE FROM Employees

WHERE emp_id = p_id;

END;

$$;
```

Call:

```
CALL delete_employee(5);
```

Outcomes:

- Learned how to create and call stored procedures for **insert, update, and delete** operations.
- Practiced using **IN, OUT, and INOUT parameters** in PL/pgSQL.
- Gained experience with **error handling** using exceptions.
- Improved efficiency and reusability of **database operations**.
- Connected stored procedures with **SQL joins** for data retrieval and analysis.